改进领域对象并接入 Svelte

一、作业背景

在 Homework 1 中，你已经围绕 `Sudoku` / `Game` 完成了一轮领域对象设计，并实现了 Undo / Redo。

但在实际提交中，出现了一个普遍问题：

> 许多同学已经“设计出了领域对象”，却没有将它真正接入现有 Svelte 游戏流程中。

结果是：

- 领域对象存在于独立模块中，但 UI 没有真正使用它
- 组件仍然直接操作旧状态或旧逻辑
- `Sudoku` / `Game` 只在测试中可用，在真实界面中没有成为核心

本次 Homework 1.1 的目标，就是解决这个问题。

二、作业目标

请在老师课堂讲解的基础上，完成以下两项任务：

任务 1：改进 Homework 1 中的领域对象

你需要根据课堂讲解与自己的反思，改进已有的 `Sudoku` / `Game` 设计。

任务 2：将领域对象接入真实 Svelte 游戏流程

你必须让现有 Svelte 前端真正消费你的领域对象，而不是只保留“独立可测试但未接入”的实现。

三、本次作业的核心问题

本次作业重点回答两个问题：

1. **Svelte 的响应式机制如何与领域对象协作？**
2. **View 层如何消费 `Sudoku` / `Game`？**

你必须在代码与文档中清楚回答这两个问题。

四、技术背景与边界

本 starter repo 当前使用的是 **Svelte 3** 风格工程，而不是 Svelte 5。
因此，本次作业应基于以下机制工作：

- store / custom store
- 组件中的 `$store`
- 顶层 `let`
- 顶层 `$:` reactive statements

本次作业**不要求**迁移到 Svelte 5，不要求使用 runes，不要求使用 reactive classes。

五、必须完成的内容

A. 改进你的 `Sudoku` / `Game`

你需要在 Homework 1 的基础上，重新审视并改进以下内容：

1. `Sudoku`

至少应承担以下职责：

- 持有当前 grid / board 数据
- 提供 `guess(...)` 或等价接口
- 提供校验能力
- 提供外表化能力（如 `toString()` / `toJSON()`）
- 能支持 Undo / Redo 所需的状态演进

2. `Game`

至少应承担以下职责：

- 持有当前 `Sudoku`
- 管理历史
- 提供 `undo()` / `redo()`
- 对外提供面向 UI 的游戏操作入口

3. 改进要求

你需要根据 HW1 的问题进行至少一项实质性改进，例如：

- 调整 `Sudoku` 与 `Game` 的职责边界
- 改进 history 的存储方式
- 改进 clone / snapshot 策略
- 改进序列化设计
- 改进对象对 UI 的暴露方式

B. 将领域对象真正接入 Svelte 真实使用流程

这是本次作业最重要的要求。

你必须让真实游戏界面中的主要流程，真正通过你的领域对象完成。

至少需要接入以下流程：

1. **开始一局游戏**
   - 创建 `Game`
   - 创建或加载 `Sudoku`
2. **界面渲染当前局面**
   - UI 中看到的 grid 必须来自你的领域对象，或来自由你的领域对象导出的响应式视图状态
3. **用户输入**
   - 用户在界面中进行输入时，必须调用你的 `Game` / `Sudoku` 接口，而不是直接操作旧数组
4. **Undo / Redo**
   - 界面中的撤销 / 重做必须调用你的领域对象逻辑
5. **界面自动更新**
   - 当领域对象发生变化后，Svelte 界面必须正确刷新

六、推荐的接入方式

你可以自行设计，但必须属于“View 真正消费领域对象”的方案，而不是“领域对象存在但 UI 没用”。

下面给出两种推荐方案。

方案 A：Store Adapter（推荐）

建立一个面向 Svelte 的适配层，例如：

- `createGameStore(...)`
- `createSudokuStore(...)`

该适配层负责：

- 持有你的 `Game` / `Sudoku`
- 对外暴露可被 Svelte 消费的响应式状态
- 对外暴露 UI 可调用的方法

例如：

- 响应式状态： `grid` ， `invalidCells` ， `won` ， `selectedCell`
- 响应式命令： `guess` ， `undo` ， `redo`

这是本次作业最推荐的方式。

方案 B：领域对象自身实现可订阅接口

你也可以让 `Game` 或 `Sudoku` 本身直接具备可订阅能力，并被组件直接消费。

但你必须自己处理好：

- 更新通知
- 订阅与取消订阅
- Svelte 的消费方式

如果你选择这种方案，必须在 `DESIGN.md` 中解释清楚其工作原理。

七、明确要求：不能这样做

以下做法**不满足**本次作业要求：

1. 领域对象只存在于测试中

例如：

- 测试里用了 `Sudoku`
- 但真实界面仍然走原先旧逻辑

这种情况视为“未接入真实使用”。

2. 只把旧数组包了一层名字

例如：

- UI 仍然直接改二维数组
- 只是顺手把它叫作 `sudoku`

这种情况不算完成领域对象接入。

3. 关键逻辑继续放在组件中

例如：

- Undo / Redo 仍主要写在 `.svelte` 文件里
- `guess` 逻辑散落在组件事件处理函数中

这种情况不符合本次作业要求。

八、关于 Svelte 响应式机制，你必须理解的事情

本次作业要求你真正理解，而不是只靠 AI “拼通”。

你至少需要理解以下问题：

1. 为什么修改对象内部字段后，界面不一定自动更新？
2. 为什么直接改二维数组元素，有时 Svelte 不会按预期刷新？
3. 为什么 store 可以被 `$store` 消费？

4. 为什么 `$:` 有时会更新，有时不会更新？
5. 为什么“间接依赖”可能导致 reactive statement 不触发？

你不一定要在作业中完整讲透所有细节，但你必须在 `DESIGN.md` 中清楚解释：

- 你的方案依赖了 Svelte 的什么机制
- 你的 UI 为什么会更新
- 如果错误地直接 mutate 对象，会出什么问题

九、AI 使用要求

本次作业**鼓励使用 AI 工具**，但有两个前提：

1. 你必须理解你提交的代码

你需要能够解释：

- 你的 `Sudoku` / `Game` 是如何被 view 层消费的
- 你的响应式更新是如何发生的
- 为什么你的写法在 Svelte 中有效

2. 你必须在文档中说明关键原理

不能只交一份“能跑”的代码，而说不清原理。

如果课堂抽查、答辩或助教提问时，无法说明技术原理，相关部分会被扣分。

十、必须提交的文档

请在仓库根目录提交 `DESIGN.md` 。

除 HW1 中已有说明外，本次必须新增以下内容：

A. 领域对象如何被消费

你必须说明：

1. View 层直接消费的是什么？
    - 是 `Game`
    - 是 `Sudoku`
    - 还是某个 store / adapter
2. View 层拿到的数据是什么？
    - `grid`
    - `invalidCells`
    - `won`
    - 其他状态
3. 用户操作如何进入领域对象？
    - 点击事件如何调用 `guess(...)`
    - Undo / Redo 如何进入 `Game`
4. 领域对象变化后，Svelte 为什么会更新？

B. 响应式机制说明

你必须说明：

1. 你依赖的是 `store` 、 `$:` 、重新赋值，还是其他机制？
2. 你的方案中，哪些数据是响应式暴露给 UI 的？
3. 哪些状态留在领域对象内部？
4. 如果不用你的方案，而是直接 mutate 内部对象，会出现什么问题？

C. 改进说明

你必须说明：

1. 相比 HW1，你改进了什么？
2. 为什么 HW1 中的做法不足以支撑真实接入？
3. 你的新设计有哪些 trade-off？

十一、建议的最低通过实现

如果你不知道从哪里下手，建议先做一个“最小正确接入版”：

第一步

保留 `Sudoku` / `Game` 作为核心领域对象。

第二步

编写一个 `createGameStore()` 或等价适配层：

- 内部持有 `Game`
- 对外暴露可订阅状态
- 对外暴露 `guess`，`undo`，`redo`

第三步

让 Svelte 组件只做两件事：

- 读取 store 中的数据来渲染
- 在事件中调用 store 暴露的方法

第四步

在 `DESIGN.md` 中解释：

- 为什么这样能工作
- 为什么 UI 能刷新

十二、评分标准

总分：100 分

1. 真实接入程度（30 分）

- 领域对象确实进入真实游戏流程
- 不是“只在测试里可用”
- 主要交互由领域对象驱动

2. 响应式正确性（25 分）

- UI 更新行为正确

- 无明显“数据变了但界面不刷新”的问题
- Undo / Redo 与渲染联动正确

3. 领域对象改进质量（20 分）

- `Sudoku` / `Game` 更清晰
- 相比 HW1 有实质性提升
- 职责边界更合理

4. View-Model / Adapter 设计（10 分）

- UI 与领域层之间的桥接清楚
- 没有严重耦合和重复逻辑

5. 文档质量（15 分)

- `DESIGN.md` 清楚解释消费方式与响应式原理
- 能说明技术选择与问题分析

十三、不要求的内容

本次作业不要求：

- 升级到 Svelte 5
- 使用 runes
- 使用 reactive classes
- 重做 UI 视觉
- 增加新功能（如 Hint / Explore）

本次唯一重点是：

> **把领域对象真正接入现有 Svelte 游戏流程。**

十四、课堂讨论准备

完成后，请准备回答以下问题：

1. 你的 view 层直接消费的是谁？
2. 为什么你的 UI 在领域对象变化后会刷新？
3. 你的方案中，响应式边界在哪里？
4. 你的 `Sudoku` / `Game` 哪些状态对 UI 可见，哪些不可见？
5. 如果将来迁移到 Svelte 5，你的哪一层最稳定，哪一层最可能改动？